

# Aruvi: Technology Partnership & Production Roadmap

From localhost prototype to production platform, mobile apps and ongoing technical support

## 1. Where Aruvi Is Today

Aruvi is not starting as a conventional software-development project. The product is already a working, extensively designed and validated prototype. The educational logic, teacher workflows and user experience have been substantially developed, while the current implementation is being used locally through localhost and has been developed with significant assistance from Claude Cowork.

The attached concept note describes Aruvi as a validated product seeking production engineering. It deliberately leaves technical architecture open and expects the technology partner to recommend the appropriate production architecture, scalability, security, deployment, technology stack, performance and operational-cost approach.

## 2. The Nature of the Relationship Needed

The appropriate relationship is therefore broader than hiring a freelancer simply to 'build an app'. Aruvi needs a technology partner who can assess the existing prototype, determine what can be retained, productionise it, establish the cloud infrastructure, launch the web application, subsequently support Android and iOS, and provide continuing technical advice and maintenance.

**Recommended relationship:** a phased, long-term technology partnership rather than a single one-off Fiverr development project.

| Phase                   | Purpose                                                                                                                                    |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Technical assessment | Review existing localhost code, architecture, data handling and integrations; identify what can be retained and what needs to be replaced. |
| 2. Productionisation    | Create the production architecture, cloud environment, database, authentication, security, deployment and monitoring.                      |
| 3. Web launch           | Deploy the responsive, mobile-first Aruvi web application and implement subscriptions and payments.                                        |
| 4. Mobile expansion     | Develop Android first and iOS subsequently, using the most appropriate technical approach.                                                 |
| 5. Ongoing support      | Maintain the live platform, monitor dependencies, handle security updates, incidents, APIs, payments and app-store listings.               |
| 6. Future development   | Add new functionality, integrations, analytics, AI capabilities and institutional features as Aruvi evolves.                               |

## 3. What the Technology Partner Should Do First

Before agreeing to a large fixed-price build, the first deliverable should be a technical assessment of the existing application. The partner should inspect the current code and establish:

- Frontend framework and overall code quality.
- Whether a backend already exists and what can be reused.
- Current data-storage arrangements and what database architecture is required.
- How AI/API calls are implemented and whether API keys or secrets are handled securely.
- Which parts of the current product can be retained without rebuilding.
- What needs refactoring or replacement for production.
- Authentication, authorization and user-data isolation requirements.
- Recommended cloud, database, deployment and monitoring architecture.

## 4. Moving from Localhost to Production

The key transition is from a locally running application to a service that teachers can access securely from their phones and computers. A typical production structure will involve the Aruvi frontend, backend/application services, cloud infrastructure, a database, authentication, storage, AI/API services and payment infrastructure.

The exact providers—such as AWS, Google Cloud, Azure, Supabase or other services—should not be selected simply because they are familiar to a freelancer. The technology partner should recommend the architecture based on Aruvi's requirements, expected scale, cost, maintainability and future roadmap.

## 5. Web, Android and iOS

The concept note proposes a responsive web application with a mobile-first experience, followed by Android first and iOS subsequently. Desktop compatibility is also important for schools, administrators and longer teacher planning sessions.

The technology partner should advise whether mobile apps should be native or use a cross-platform framework, and whether any part of the existing web implementation can be leveraged. There is no need to assume that three separate codebases must be created.

## 6. Payments and Platform Fees

Payment architecture should be treated as a strategic design decision rather than a simple gateway integration. Aruvi will initially operate on individual teacher subscriptions, with institution-level licensing as a future direction.

The technology partner should specifically advise on the optimal payment architecture across the web, Android and iOS, including applicable Apple App Store and Google Play requirements, platform fees, subscription mechanics and the implications for Aruvi's unit economics. The partner should also recommend appropriate payment gateways for the Indian market.

## 7. Ongoing Maintenance and External Dependencies

Once Aruvi is live, technical responsibility does not end when the initial development is completed. The platform will depend on third-party services such as cloud infrastructure, databases, authentication, payment providers, AI APIs, email services, app stores and software libraries. These dependencies can change or fail independently of Aruvi's own code.

- Cloud or database outages and performance problems.
- Security vulnerabilities in software dependencies.
- Changes to third-party APIs or authentication systems.
- Payment-gateway changes or failures.
- Apple or Google app-store policy and technical changes.
- AI-provider API changes, pricing changes or service interruptions.
- Backups, disaster recovery and restoration.
- Production bugs, monitoring alerts and incident response.

For this reason, a continuing monthly support/maintenance arrangement is likely to be more appropriate than treating the relationship as a one-time development engagement.

## 8. Security and Data Responsibility

Security should be an explicit responsibility of the technology partner. The production system should address authentication, authorization, user-data isolation, database permissions, API security, encryption, secure management of secrets, backups, logging, dependency vulnerabilities, secure deployment and appropriate

protection against common web attacks.

This is especially important because Aruvi requires individual teacher accounts and secure isolation of each teacher's data, while leaving room for future institutional structures.

## 9. Ownership and Vendor Independence

A critical principle is that Aruvi—not the external developer—should own and control the core infrastructure and accounts. Wherever possible, the following should be created in Aruvi's own organisational accounts:

- Source-code repository / GitHub.
- Cloud infrastructure account.
- Database and storage account.
- Domain and DNS.
- Apple Developer account.
- Google Play Developer account.
- Payment gateway account.
- AI/API accounts.
- Analytics and monitoring services.

The developer should receive appropriate access rather than becoming the owner of these resources. This substantially reduces vendor lock-in and makes it possible to change technical partners later.

## 10. Handover Requirements

At the end of the initial production engagement, Aruvi should receive complete operational control and documentation, including:

- Source code and repository access.
- Deployment and infrastructure documentation.
- Database schema and configuration documentation.
- API and integration documentation.
- Environment and secrets-management documentation.
- Backup and disaster-recovery procedures.
- App-store accounts and submission materials.
- Payment integration documentation.
- A list of third-party dependencies and services.

A useful test is: another competent developer should be able to take over the system without needing the original developer to explain how it works.

## 11. Recommended Commercial Structure

A sensible commercial model would be to separate initial development from ongoing support:

- **Initial technical assessment:** a defined, relatively small engagement resulting in an architecture and production plan.
- **Production build:** milestone-based development with clearly defined deliverables and acceptance criteria.
- **Launch:** deployment, testing, security checks and web/app-store release.

- **Post-launch support:** monthly retainer covering maintenance, monitoring, security and third-party dependency issues.
- **New features:** separately scoped development work as the product evolves.

## 12. Suggested Fiverr Brief

Please review the attached concept note carefully, as it provides the background, product vision and current state of Aruvi. We have a working and extensively validated prototype, currently developed and tested locally, and are now looking for a technology partner to help us take it to production. This would include assessing the existing code and advising what can be retained, setting up the appropriate cloud/backend/database infrastructure, launching the responsive web application, and subsequently developing Android and iOS apps. We would particularly value advice on architecture, security, authentication, scalability, payment gateways and the optimal payment structure across web, Android and iOS, including Apple/Google platform requirements and fees. We are not looking only for someone to code to a fixed specification; we need a partner who can advise us on technical choices and continue to support the platform after launch, including database/cloud issues, security updates, third-party dependencies, API changes, payment issues and app-store requirements. Please explain how you normally structure such relationships—for example, an initial milestone-based development engagement followed by a monthly maintenance/support or advisory arrangement—and how ownership and access to the source code, cloud infrastructure, databases and third-party accounts are handled.

## 13. Key Principle

**You have already built much of the product.** The external technology partner is not being hired to invent Aruvi. Their principal value is to convert the existing product into a secure, maintainable, commercially deployable and scalable service—and then help you operate it reliably.